



**apex-bancard-pos-plugin**

## **POS Bancard – Cobro directo (cliente)**

Plugin de Dynamic Action para Oracle APEX: cobra con un terminal físico POS Bancard directo desde el navegador del cajero, sin ningún servidor de aplicaciones en el medio. Genérico, instalable en cualquier app APEX.

Oracle APEX 20.2 / 24.x (verificado)

Dynamic Action plugin

Licencia MIT

## 1 Por qué existe y cómo funciona

Una integración servidor-a-servidor con un terminal POS Bancard funciona solo **si el servidor de aplicaciones llega por red a la IP local del terminal**. En muchos despliegues reales el servidor y el terminal están en segmentos de red distintos, y ese camino nunca conecta.

Este plugin es la vía alternativa: quien sí está en la misma LAN que el terminal es **el navegador del cajero**, no el servidor. El plugin hace `fetch()` directo desde el navegador al IP/puerto local del POS — sin ningún backend en el medio.

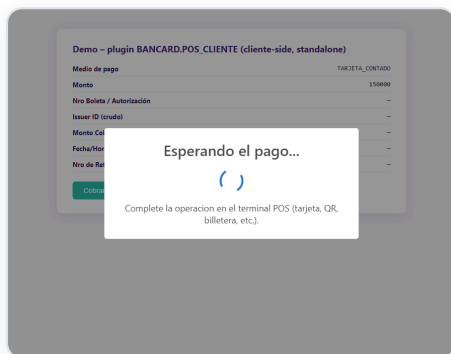
**Coexiste, no reemplaza.** Pensado para funcionar junto a una integración servidor-a-servidor existente, si ya tenés una: usá esa vía donde el servidor llegue al POS, y este plugin donde no llegue. Ambos caminos pueden convivir en la misma página.

### Flujo de ejecución

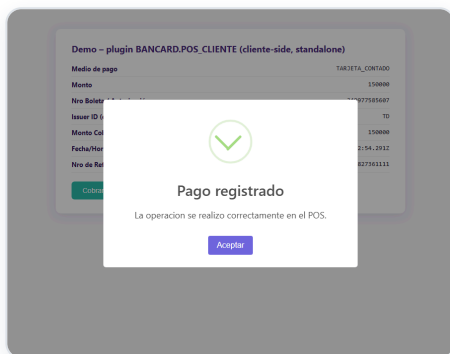
- 1 El cajero dispara la Dynamic Action (por ejemplo, el click de un botón "Cobrar con POS").
- 2 El JavaScript del plugin hace `POST /pos/eco` al terminal (timeout 5s) para confirmar que está despierto.
- 3 Según el **Medio de Pago** configurado, llama al endpoint correspondiente del terminal (timeout 90s, pensado para dar margen real a que el cliente complete la operación).
- 4 Escribe el resultado **crudo** del POS en los items destino configurados, disparando su evento `change` (no lo suprime) para que la app pueda reaccionar.
- 5 Todo el feedback visual —loader, éxito, error— se muestra con **SweetAlert2**, empaquetado con el plugin (sin CDN externo, sin depender de `apex.message`).

No hay ningún round-trip a un servidor de aplicaciones en el medio: todo el protocolo Bancard corre en el navegador. El número de referencia ( `facturaNro` ) se genera con `Date.now()`, sin depender de ninguna secuencia de base de datos.

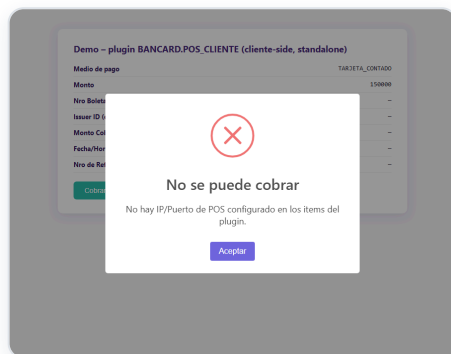
### Feedback: siempre modal de SweetAlert2



Cargando — spinner armado a mano en el `html` del modal (no depende de `Swal.showLoading()`, ver nota abajo). Nunca muestra ningún botón.



Éxito — modal con ícono verde y botón "Aceptar", no un toast que desaparece solo.



Error — mismo patrón para cualquier falla: validación de items, rechazo del POS, timeout, o problema de red/CORS.

**Nota técnica — por qué el loader no usa `Swal.showLoading()`**. Ese método de SweetAlert2 funciona convirtiendo el botón "Confirmar" en el spinner; si no hay un botón visible para convertir ( `showConfirmButton:false` ), el spinner no tiene dónde insertarse. Y si el swap no se revierte bien antes de la siguiente alerta, el botón "Confirmar" puede quedar

escondido para siempre. En vez de depender de ese acople, el plugin arma el spinner a mano en el `html` del modal (reusando la clase CSS `.swal2-loader` del bundle) con `showConfirmButton:false` fijo: nunca hay botón durante la carga, y nunca hay nada que revertir.

**Requisito obligatorio del navegador (solo contra un terminal real).** El terminal Bancard habla HTTP plano y **no manda headers CORS en sus respuestas** — confirmado contra hardware real y contra la documentación oficial del protocolo (ningún endpoint declara ni valida un header propio). Esto genera **dos** restricciones de navegador distintas, y hay que resolver las dos. **No hace falta nada de esto contra el simulador incluido** (sección 5): ya responde los headers CORS correctos.

**1. Acceso a red local / contenido mixto** — si APEX se sirve por HTTPS, la solicitud puede bloquearse antes de salir. Click en el candado de la barra de direcciones → **Configuración de sitios** → habilitar **"Red local"** y **"Contenido no seguro"** para ese sitio.

**2. CORS en la respuesta** — esta parte **ningún permiso de sitio la resuelve**. El navegador manda la petición igual (el terminal la recibe y la procesa), pero al no traer la respuesta el header `Access-Control-Allow-Origin`, el navegador le prohíbe al JavaScript leerla — el `fetch()` falla aunque la operación ya se haya ejecutado en el terminal. Pasa siempre en un pedido cross-origin, con o sin preflight; ninguna configuración de sitio lo desactiva. Dos soluciones reales: instalar y **activar explícitamente** una extensión tipo "Allow CORS" (apagarla cuando no se esté cobrando — mientras está activa afecta a *todos* los sitios que visite esa máquina), o un proxy local en esa misma máquina que agregue el header faltante (más robusto para varios puestos de cobro, no incluido todavía en este repo).

## 2 Instalación

### Recomendado: exports reales verificados

`install/install_plugin_apex20.sql` e `install/install_plugin_apex24.sql` **no son scripts escritos a mano** — son exports reales de Oracle (Shared Components → Plugins → Export), con los datos identificatorios de origen reemplazados por placeholders. El resto (IDs internos, los 11 atributos, el JS empaquetado) es exactamente el export real, byte a byte.

Esto está verificado en la práctica: el de 24.2 se generó **después** de importar el export de 20.2 en una aplicación APEX completamente distinta —otra instancia, otro workspace, otra app— vía Import File, sin tocar nada a mano, y funcionó sin problema.

- 1 Editar los 3 valores marcados `REPLACE` al principio del script que corresponda a tu versión ( `p_default_workspace_id` / `p_default_application_id` / `p_default_owner` ).
- 2 Correr en **SQL Workshop** → **SQL Scripts** (no SQL Commands: el script supera el límite de 32 KB).
- 3 Verificar: **Shared Components** → **Plug-ins** debe mostrar "POS Bancard – Cobro directo (cliente)".

**Por qué esto es seguro y escribir el script a mano no lo es.** Un export real usa un ID interno de 15-20 dígitos para el plugin, generado por la secuencia real de la instancia — al reimportarlo, ese número prácticamente no puede colisionar con nada. Un script escrito a mano con un ID chico inventado (ej. `wwv_flow_api.id(9201)` ) sí puede colisionar con algún objeto real y viejo de la instancia — y como la metadata de plugins se carga a nivel de toda la aplicación, no por página, una sola colisión puede romper Page Designer para **toda la app**, en cualquier página, para cualquier desarrollador. No es una hipótesis: pasó exactamente así en el desarrollo de este plugin, y se resolvió desinstalando con `WWV_FLOW_API.REMOVE_PLUGIN` y reinstalando con un export real.

### Alternativa: por la UI de APEX

**Shared Components** → **Plug-ins** → **Create Plugin**, cargando los valores a mano. Guía completa con todos los textos exactos para copiar/pegar: `install/manual_ui_install.md` . Mismo resultado que un export real: APEX genera un ID propio, sin riesgo de colisión.

### Instalar en otra app / otro workspace

Una vez instalado, la vía estándar de APEX: **Shared Components** → **Plug-ins** → abrir el plugin → **Export**, y en la app destino **Shared Components** → **Plug-ins** → **Import File**. APEX resuelve el workspace/app/owner destino automáticamente y genera un ID real.

### 3 Cómo integrarlo en una página de cobro

El patrón que funciona de punta a punta tiene **tres piezas** en la misma Dynamic Action:

#### 1. Resolver configuración

Acción nativa *Execute Server-side Code*: PL/SQL con bind variables que resuelve IP/Puerto/Medio de Pago/Monto según la lógica propia de tu app, escribiéndolos en los items de configuración del plugin.

#### 2. Acción del plugin

La Dynamic Action del plugin en sí (*POS Bancard - Cobro directo (cliente)*), con los 11 atributos mapeados a los items de tu página.

#### 3. Mapear el resultado

Un evento **Change** separado (no una acción encadenada) sobre uno de los items destino, que copia el resultado crudo a tu formulario real.

**Por qué la pieza 3 es un evento separado, no una tercera acción encadenada.** La acción del plugin corre de forma asíncrona sin avisarle al motor de Dynamic Actions que espere — para APEX, "termina" apenas se dispara, no cuando el POS responde. Una acción encadenada después se ejecutaría de inmediato, antes de tener resultado. El plugin no suprime el evento `change` al setear sus items destino, así que un evento Change separado sobre uno de ellos siempre dispara una vez que el resultado realmente está.

#### Los 11 atributos del plugin

| #  | Atributo                                 | Contenido del item                                             | Oblig. |
|----|------------------------------------------|----------------------------------------------------------------|--------|
| 1  | Item: IP del POS                         | IP local del terminal                                          | Sí     |
| 2  | Item: Puerto del POS                     | Puerto del terminal                                            | Sí     |
| 3  | Item: Medio de Pago                      | Código del medio (sección 4)                                   | Sí     |
| 4  | Item: Monto                              | Monto a cobrar, número plano                                   | Sí     |
| 5  | Item: Datos Adicionales (JSON)           | JSON según el medio — sección 4                                | No     |
| 6  | Item destino: Nro Boleta/Autorización    | <code>nroBoleta</code> / <code>codigoAutorizacion</code> crudo | Sí     |
| 7  | Item destino: Issuer ID                  | <code>issuerId</code> crudo — sin mapear                       | Sí     |
| 8  | Item destino: Monto Cobrado              | Monto cobrado, número plano                                    | Sí     |
| 9  | Item destino: Fecha/Hora de la Operación | ISO 8601                                                       | Sí     |
| 10 | Item destino: Nro de Referencia          | <code>facturaNro</code> generado                               | Sí     |
| 11 | Item destino: Resultado Completo (JSON)  | Respuesta cruda completa                                       | No     |

#### Pieza 1 — PL/SQL de ejemplo

**Bind variables, no `apex_application.g_x01`.** "Execute Server-side Code" lee/escribe items con `:ITEM` directo (vía "Items to Submit"/"Items to Return"), y no espera ni imprime JSON con `sys.http.p`. Ese otro patrón es para un **proceso de página Ajax Callback** invocado a mano desde JavaScript — mecanismo distinto, sintaxis distinta. Mezclarlos hace que la acción no setee nada, sin ningún error visible.

```

declare
    l_ip_pos      mi_tabla_terminales.ip_pos%type;
    l_puerto_pos  mi_tabla_terminales.puerto_pos%type;
begin
    select t.ip_pos, t.puerto_pos into l_ip_pos, l_puerto_pos
    from mi_tabla_terminales t
    where t.id_sucursal = :P_ID_SUCURSAL
        and t.puesto     = :P_PUESTO
        and t.activo      = 'S';

    :P_POS_IP      := l_ip_pos;
    :P_POS_PUERTO  := to_char(l_puerto_pos);
    :P_POS_MEDIO_PAGO := case :P_TIPO_TARJETA
                        when 4 then 'TARJETA_DEBITO'
                        else 'TARJETA_CONTADO'
                        end;
    :P_POS_MONTO_PLANO := trim(replace(:P_MONTO, '.', ''));
exception
    when no_data_found then
        raise_application_error(-20001, 'No hay POS Bancard configurado para este puesto de cobro.');
```

### Pieza 3 — mapeo del resultado (ejemplo)

```

function formatMiles(n) {
    n = Math.round(Number(n) || 0);
    return n.toString().replace(/\B(?=(\d{3})+(?!\d))/g, '.');
}
apex.item("MI_ITEM_NRO_CHEQUE_TARJETA").setValue(
    formatMiles(apex.item("P_POS_NRO_BOLETA").getValue())
);
apex.item("MI_ITEM_MONTO").setValue(
    formatMiles(apex.item("P_POS_MONTO_COBRADO").getValue())
);
```

## 4 Medios de pago y mapeo aguas abajo

### Medios de pago soportados

| Código (Item: Medio de Pago) | Endpoint(s) Bancard               | Datos Adicionales (JSON)                         |
|------------------------------|-----------------------------------|--------------------------------------------------|
| TARJETA_CONTADO              | venta-ux → descuento              | —                                                |
| TARJETA_CUOTAS               | venta-ux (con cuotas) → descuento | {"cuotas":N,"plan":N}                            |
| TARJETA_DEBITO               | venta/debito → descuento          | —                                                |
| TARJETA_CREDITO              | venta/credito → descuento         | {"cuotas":N,"plan":N}                            |
| QR                           | venta-qr                          | {"montoVuelto":N,"promotions":[...]} (opcional)  |
| QR_PIX                       | venta-qr-pix                      | {"pix_payer_cpf":"...", "pix_payer_phone":"..."} |
| EXTRACCION_QR                | extraccion-qr                     | —                                                |
| CANJE                        | venta-canje                       | —                                                |
| CANJE_QR                     | venta-canje-qr                    | —                                                |
| BILLETERA                    | venta-billetera                   | {"billetera":"ZIM","cuenta":"123456"}            |

`anulacion` / `consulta-anulacion` quedan fuera del plugin a propósito: son operaciones de gestión post-venta, no medios de pago.

### Mapeo aguas abajo (a cargo de cada app)

Cada empresa mantiene su propio catálogo de marcas de tarjeta. El plugin no lo conoce; el mapeo se hace en la página, típicamente en el mismo evento "Change" de la pieza 3:

```
var marcaPropia = MI_CATALOGO_MARCAS[ apex.item('P_POS_ISSUER_ID').getValue() ] || 99;
apex.item('MI_ITEM_MARCA_TARJETA').setValue(marcaPropia);

// Lo mismo aplica a formato de fecha: convertir P_POS_FECHA (ISO 8601)
// al formato que tu proceso de guardado espere.
```

Este mapeo es intencionalmente responsabilidad de cada app, no del plugin: es lo que permite instalar el mismo plugin, sin modificarlo, en cualquier aplicación con su propio catálogo de marcas y su propio formato de guardado.

## 5 Probar sin terminal físico

`simulator/pos_simulator.py` implementa los 13 endpoints reales del protocolo Bancard v1.5.0, corre en la misma máquina o en la LAN, sin dependencias externas:

```
python pos_simulator.py --port 3000
```

### Simular realismo: demora y resultados aleatorios

| Flag                                              | Qué hace                                                                                                                                                           |
|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--delay-cliente N</code> o <code>N-M</code> | Demora simulando el tiempo que tarda el cliente en completar la operación. Número fijo o rango aleatorio. Se aplica a todo excepto <code>/pos/eco</code> .         |
| <code>--random</code>                             | En vez de aprobar siempre, rechaza ventas al azar (fondos insuficientes, tarjeta vencida/bloqueada, etc.), hace timeout, o falla el propio <code>/pos/eco</code> . |
| <code>--fail-rate</code>                          | Probabilidad (0-1) de rechazo. Default 0.15.                                                                                                                       |
| <code>--timeout-rate</code>                       | Probabilidad (0-1) de timeout. Default 0.03.                                                                                                                       |
| <code>--eco-fail-rate</code>                      | Probabilidad (0-1) de que falle el <code>/pos/eco</code> . Default 0.05.                                                                                           |
| <code>--interactive</code>                        | Aprobar/rechazar/timeout cada venta por consola, para reproducir un caso puntual.                                                                                  |

```
python pos_simulator.py --port 3000 --delay-cliente 5-10 --random
```

**Ojo con el margen entre el delay simulado y el timeout del plugin.** El timeout de operación del plugin es de 90 segundos. Si el máximo de `--delay-cliente` está cerca o por encima de eso, el navegador puede reportar `Failed to fetch` en vez de completar la prueba — dejar el rango bien por debajo del timeout.

Referencia completa de endpoints y flags: `simulator/README.md`. Para probar el plugin solo, sin ninguna app APEX: `js/demo.html` — formulario de configuración a la izquierda, mock interactivo del terminal a la derecha, reflejando en vivo cada llamada real al simulador.



| Síntoma                                                                                  | Causa probable                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>net::ERR_EMPTY_RESPONSE</code> contra un terminal <b>real</b>                      | Preflight CORS — confirmar que el plugin usa <code>Content-Type: text/plain</code> , no <code>application/json</code> (el terminal no responde <code>OPTIONS</code> ).                                                              |
| Consola: <code>blocked by CORS policy: No 'Access-Control-Allow-Origin' header...</code> | El pedido llegó y el terminal lo procesó, pero el navegador bloqueó la lectura de la respuesta — activar la extensión CORS o el proxy local, no es un problema de IP/puerto.                                                        |
| <code>Failed to fetch</code> / mixed content contra un terminal <b>real</b>              | Falta habilitar "Red local" + "Contenido no seguro" para el sitio (candado → Configuración de sitios) en esa máquina.                                                                                                               |
| <code>Failed to fetch</code> contra el <b>simulador</b>                                  | <code>--delay-cliente</code> muy cerca del timeout del plugin (90s) — bajar el rango del delay.                                                                                                                                     |
| "El POS no respondió dentro del tiempo de espera"                                        | Terminal apagado, IP/puerto mal configurados, o (contra el simulador) salió el resultado aleatorio de timeout con <code>--random</code> .                                                                                           |
| Items de configuración llegan vacíos a la acción del plugin                              | "Execute Server-side Code" (pieza 1) escrita con <code>g_x01 / sys.http.p</code> en vez de bind variables — ver sección 3. No tira error, simplemente no setea nada.                                                                |
| La acción del plugin encadenada tras otra acción async nunca recibe atributos            | Mover el mapeo del resultado a un evento "Change" separado (pieza 3), no una tercera acción encadenada.                                                                                                                             |
| El botón de cobro no aparece/desaparece en vivo                                          | Una condición server-side (PL/SQL) se evalúa solo al renderizar — agregar tu propia Dynamic Action de mostrar/ocultar ligada al cambio de los items relevantes.                                                                     |
| Rompió Page Designer para toda la app                                                    | Plugin instalado con un ID chico puesto a mano vía <code>wwv_flow_api</code> en vez de un export real o la UI — ver sección 2. Desinstalar con <code>WWV_FLOW_API.REMOVE_PLUGIN</code> y reinstalar con un export real o por la UI. |

## 7 Compatibilidad de versiones de APEX

Desarrollado y probado en APEX 20.2, y **verificado en la práctica** contra una instancia 24.2 completamente distinta: el plugin se importó vía Export/Import sin tocar nada y funcionó de punta a punta.

- **La API que usa el `render` del plugin** ( `apex_plugin.t_dynamic_action` / `t_dynamic_action_render_result` ) es estable y documentada: los 11 atributos que usa este plugin no cambiaron entre 20.1 y 24.2.
- **Lo que sí cambió: el paquete interno que usa el propio export/import de Oracle.** El export real de una instancia 20.2 usa `wwv_flow_api.*` ; el de una instancia 24.2 usa `wwv_flow_imp.*` / `wwv_flow_imp_shared.*` — paquetes distintos, con al menos un parámetro que desapareció ( `p_supported_ui_types` ) y uno nuevo que apareció ( `p_version_scn` ). No afecta al plugin en uso, pero si algún día necesitas tocar el script de instalación a mano, no asumas que la firma es idéntica entre versiones.
- **SweetAlert2 va empaquetado como archivo propio del plugin**, no cargado desde un CDN externo — pensado para instancias con Content Security Policy reforzada (documentado por Oracle para APEX 24.2).
- **APEX 24.1 deprecó "Substitute Attribute Values"** para plugins de región; no afecta a este plugin, que usa el patrón más simple de Dynamic Action (solo `p_render_function` ).

Instalando con los exports reales o por la UI (sección 2), la compatibilidad de versión la resuelve APEX solo.